

Hybrid AI Tutoring System

Architecture, Request Flow & Team Implementation Context

Member Working Document

Purpose. This document reorganizes the existing architecture discussion into one coherent reference. It explains what the system is, what each tier does, how a student request moves through the system, how tutors and administrators access information, and how feedback is used to improve the tutor.

Important. This is a cleaned-up version of the supplied planning material. It preserves the decisions, terminology, examples, and open questions in that material rather than introducing a new architecture.

1. System Overview

The proposed system is a Hybrid AI Tutoring Platform designed to support students across both theory-based and programming/coding modules without simply giving them final answers. The central idea is a Socratic tutoring workflow: the student asks an academic question or submits a coding problem, the system retrieves the relevant course material, the AI generates guided tutoring help, and a guardrail checks the response before it reaches the student. For the current system scope, the platform supports four modules within one course.

The architecture is divided into four major tiers:

- Client Tier — the interfaces used by students, tutors, and administrators.
- Orchestration Tier — the backend workflow that authenticates requests, determines intent, applies tutoring rules, retrieves course context, and checks AI output.
- Hybrid Data Tier — the structured PostgreSQL data and pgvector-based vector data used for users, module access, telemetry, curriculum material, and code-repair patterns.
- Inference Tier — the open-weight generative model used to produce the actual tutoring response. For the current system, inference is intended to run locally through Ollama rather than through a production cloud deployment.

1.1 Core Design Principle

The AI should behave as an academic tutor rather than an answer generator. This applies to both theory and coding: the system should guide students toward understanding, not simply produce completed answers or solutions. The system therefore separates the student's request from the final generation step and places explicit pedagogical rules, course context, and output checks around the model.

2. Client Tier: What Users See

The Client Tier contains three user-facing roles: Student, Tutor, and Admin. Each role has a different level of access.

Role	Main Interface	Purpose
Student	Student Chat	Ask questions, submit code, receive Socratic hints, and provide feedback.
Tutor	Tutor Dashboard	View analytics and struggle trends only for assigned modules.
Admin	Admin Dashboard	Manage tutor-module access and monitor system-wide AI performance.

2.1 Student Chat Window

The Query & Code Input and the Socratic Hint Viewer are components of the same chat window. The student types a conceptual question or pastes broken code into the input area, while the conversation and AI responses appear in the scrolling chat area.

- Module selection: the student selects the module they are currently studying.
- Query & Code Input: accepts theory/conceptual questions, problem-solving questions, and coding/debugging input where relevant to the selected module.
- Socratic Hint Viewer: displays the audited AI response.
- Hint Feedback: the student can rate an AI response with thumbs-up or thumbs-down.

2.2 Module-Based Chats

The module dropdown is not only a UI convenience. The selected module becomes part of the session context. A chat started for Software Development Fundamentals remains separated from chats for other subjects.

- Protects the conversation context from unrelated subjects.
- Allows each chat session to carry a `session_id` and `module_id`.
- Makes tutor analytics easier because activity can be grouped by module.
- Keeps module selection explicit while automatic assignment from official enrollment data is deferred for the current scope.

3. API Gateway: The System's Front Door

The API Gateway sits between the Client Tier and the Orchestration Tier. Every important request reaches the Gateway first. It acts as the traffic controller, authentication checkpoint, and routing layer.

1. Receive the request from the student, tutor, or admin interface.
2. Verify the user's session and identity.
3. Confirm the relevant role and applicable module permissions.
4. Identify the type of request.
5. Route a student question into the AI processing pipeline.
6. Route feedback directly to telemetry rather than sending it through the AI agents.

Authentication is part of the final system design, not a temporary prototype-only feature. Both students and tutors will sign in using their institutional DUT4life email accounts. The application should delegate identity verification to DUT's Microsoft-backed identity/login system rather than storing or managing DUT passwords itself. After authentication, the application applies its own role and module permissions: Students receive student access, while Admins grant tutor privileges and assign tutors to specific modules. The exact DUT identity-provider/tenant application-registration details must be confirmed during implementation.

4. Users, Modules & Access Control

The system uses module-based access control. This is important because the same concept can appear in different subjects. For example, the word 'class' could refer to object-oriented programming in C# or a different concept in another module.

4.1 Student Module Scope

The student's active module is used as a metadata filter. For the current scope, the student selects one of the four supported modules from the module dropdown, and the selected `module_id` becomes part of the session. The RAG Orchestrator searches only vector records tagged with that module. Automatic assignment of modules from a student's official enrollment record is a future enhancement and is deliberately not a priority for the current four-module, single-course scope.

4.2 Tutor Module Scope

Tutors must only see the modules they are assigned to. This should be enforced at the database/query level, not merely hidden in the frontend.

A `tutor_assignments` mapping links `tutor_id` to `module_id`. When a tutor requests dashboard analytics, the backend filters the results according to that mapping. This prevents data from unrelated modules from reaching the tutor's client.

4.3 Admin Scope

The Admin has global oversight. The Admin grants tutor privileges, manages tutor-module mappings, and monitors system-wide AI performance across the four supported modules.

4.4 Authentication & Privilege Assignment

The login identity comes from the student or tutor's DUT4life institutional account. The application then maps the authenticated identity to one of the three system roles: Student, Tutor, or Admin. Tutor access is not granted merely because someone can log in with a DUT4life account; an Admin must explicitly grant Tutor privileges and assign the tutor to one or more modules. This separates authentication (who the person is) from authorization (what that person is allowed to access).

5. Orchestration Tier: The Core Backend Workflow

The Orchestration Tier is the central processing layer. It takes the authenticated student request, determines what the student needs, applies the tutoring rules, retrieves relevant curriculum context, sends the final prompt to the inference model, and checks the model's response.

5.1 Components

Component	Role	Main Responsibility
API Gateway	Front door	Authentication, routing, and access control.
Intent Agent	Classifier	Determines what the student is trying to do and validates scope.
Scaffolding Engine	Socratic rule-maker	Controls how the AI should teach rather than simply answer.
RAG Orchestrator	Librarian	Retrieves relevant, module-specific course context and builds the prompt.
Guardrail Agent	Final checker	Checks the generated response before delivery and logs failures.

6. Intent Agent

The Intent Agent determines what the student is trying to accomplish across both theory and coding contexts. Examples include a conceptual question, a request to work through an academic problem, a debugging request where coding is relevant, or an attempt to bypass the tutoring rules.

- Conceptual question: 'What is a for loop?'
- Debugging request: 'Why is my C# code throwing an IndexOutOfRangeException error?'
- Potential bypass request: 'Write my assignment for me.'

The Intent Agent also works with the student's module scope. The backend can verify that the student is enrolled in the selected module before allowing the request to continue.

7. Scaffolding Engine: The Pedagogical Brain

The Scaffolding Engine is responsible for enforcing the tutoring philosophy. Instead of sending the student's question directly to the LLM, the engine adds hidden instructions that tell the model how to guide the student.

7.1 Socratic Progression

7. Question
8. Hint
9. Student Attempt
10. Feedback
11. Further Guidance
12. Explanation

The Flask backend tracks the state of the current problem. A first request can begin at the Hint stage. If the student responds with an attempted correction, the system can advance to Student Attempt and change the instructions given to the model. Later stages can provide increasingly direct guidance.

7.2 Prompt Rules

A simplified example of the hidden instruction is:

You are a peer tutor. Guide the student step-by-step. Ask a leading question. Do not write the final code for them.

7.3 Few-Shot Prompting

The engine can strengthen these instructions by including examples of the desired tutoring behaviour. The backend dynamically constructs the prompt using the rules, examples, current student input, and current state.

7.4 Preventing Scaffolding Collapse

A major concern is that a student may repeatedly ask for the exact answer. The Scaffolding Engine therefore acts as a hard pedagogical constraint. Unless the session reaches the appropriate final explanation stage, the model should not provide a complete copy-paste solution.

8. RAG Orchestrator: The Course-Material Librarian

RAG stands for Retrieval-Augmented Generation. The RAG Orchestrator prevents the model from relying only on its general training knowledge by retrieving relevant course material and adding it to the prompt.

8.1 Why RAG Is Needed

The goal is for the tutor to use the terminology, rules, examples, and explanations contained in the official course material. This helps keep responses aligned with the syllabus and reduces unsupported answers.

8.2 RAG Workflow

13. Receive the student's question, session history, and `module_id`.
14. Convert the student's text into an embedding vector using a dedicated embedding model.
15. Apply the module metadata filter.
16. Perform semantic similarity search in the pgvector store.
17. Retrieve the most relevant curriculum chunks and, where applicable, module-specific code-repair patterns.
18. Assemble the tutoring rules, retrieved context, chat history, and current question.
19. Send the assembled prompt to the Inference Tier.

8.3 Metadata Filtering

Each stored curriculum chunk is tagged with metadata such as its `module_id`. The module filter is applied before the similarity search so that material from other modules is excluded.

Example: if the active module is Software Development Fundamentals (C#), the vector search should return C# curriculum chunks tagged for that module rather than similarly worded material from another subject.

8.4 Example pgvector Query

```
SELECT chunk_text, section_title
FROM curriculum_embeddings
WHERE module_id = 'software_dev_101'
ORDER BY embedding <=> '[query_vector_array_here]'
LIMIT 3;
```

9. Embeddings vs. the Generative LLM

An embedding model and a generative LLM both use neural-network technology, but they perform different jobs.

Model Type	Input	Output / Purpose
Generative LLM	Student prompt + context	Generates the tutoring response.
Embedding model	Student question or document text	Produces numerical vectors representing semantic meaning.

The supplied architecture discussion proposes using Ollama for the inference side while also running a dedicated embedding model for retrieval. The embedding model should be lightweight so that it does not unnecessarily compete with the main generation model for resources.

10. Model Strategy

The planning discussion considered several model options. For the current system, the recommended starting generative model is Qwen3 4B, run locally through Ollama. It is an open-weight model and is small enough to be practical for the project's local testing environment while still supporting general academic tutoring and coding tasks. The important architectural decision remains that the generative model can be changed without changing the rest of the system.

10.1 Local Development

The earlier discussion considered several lightweight models because the system is intended to run locally. The final direction is not restricted to coding: the selected model must handle general academic explanation as well as programming assistance. No production deployment or mandatory cloud inference is planned for the current project.

10.2 Inference Tier Decision

The current inference choice is Qwen3 4B through local Ollama. This replaces the earlier Qwen2.5-7B-Instruct discussion as the working model choice for the present system. The architecture should still expose the model behind a replaceable inference interface so another open-weight model can be evaluated later without redesigning the rest of the pipeline.

The key point for the team is that Qwen3 4B is the current working choice, not a permanent architectural dependency. The system should be designed so that the inference model can be replaced and tested without changing the Gateway, Scaffolding, RAG, Guardrail, database, or dashboard layers.

10.3 Embedding Model

The recommended embedding model for the current system is nomic-embed-text, running locally through Ollama. It is used only for converting student queries and curriculum chunks into vectors for RAG; it does not generate the tutoring response. The team should still validate retrieval quality on the actual module material, but nomic-embed-text is the working embedding choice for the system.

11. Guardrail Agent: Final Response Check

The Guardrail Agent receives the LLM's draft before it is delivered to the student. Its purpose is to check that the response follows the tutoring rules and remains within the intended course scope.

- Check for large blocks of copy-pasteable code where coding is involved, and check for equivalent complete-solution leakage in theory-based responses.
- Detect possible full-solution leaks.
- Identify obvious responses outside the retrieved course context.
- Block or truncate a response when it violates the system's constraints, especially when it leaks a complete solution or produces output outside the intended retrieved course context.
- Record relevant failure flags and sequence depth for telemetry.

The Guardrail therefore acts as the final gate between the model and the student interface.

11.1 Guardrail Rules Supported by the Study

The full paper establishes the Guardrail as a final audit step intended to block leaked complete answers and keep generated responses within the retrieved curriculum context. It does not provide numerical thresholds for what counts as a large code block, how many lines may be returned, or an exact truncation percentage. Those thresholds should therefore be treated as an implementation/evaluation decision rather than presented as an existing requirement from the research paper. The non-negotiable behaviour is the protection against complete-solution leakage and materially out-of-scope responses.

12. Student Feedback & LLM Health

Students can rate individual AI responses with thumbs-up or thumbs-down. Thumbs-down reason tags are included so negative feedback is actionable rather than being only a binary signal.

12.1 Downvote Reason Tags

- Too confusing / still stuck — the student did not understand the guidance.
- Too long / too short — the response format was not useful.
- Gave away answer / incorrect answer or code — possible guardrail or accuracy problem.

The reason tags are part of the current feedback design. The basic interaction remains a thumbs-up/thumbs-down rating, with the reason tags giving the Admin and Tutors more useful information about why a response was negatively rated.

12.2 Admin-Level Metrics

- Satisfaction rate across modules.

- Breakdown of failure reasons using the included thumbs-down reason tags.
- Abandoned sessions after negative feedback.
- Number of turns required to resolve a problem.
- Guardrail/code-leak flags.
- System-wide changes in negative feedback.

These metrics help distinguish a difficult academic topic from a system-wide AI problem. A tutor may see a problem limited to their module, while the Admin can identify failures appearing across multiple modules.

13. Telemetry Data

The feedback and interaction data should be stored in the relational data tier so that it can be linked to the student session, module, AI message, and time of interaction.

```
CREATE TABLE message_feedback (  
  feedback_id SERIAL PRIMARY KEY,  
  session_id VARCHAR(64) NOT NULL,  
  module_id VARCHAR(32) NOT NULL,  
  message_id VARCHAR(64) NOT NULL,  
  rating SMALLINT NOT NULL, -- +1 thumbs up, -1 thumbs down  
  reason_tag VARCHAR(32), -- optional  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

14. Hybrid Data Tier

The data layer is split conceptually into structured relational data and vector-based data. The two work together.

Data Area	Contains	Main Use
Relational Store	Users, module enrollments, tutor-module mappings	Authentication, authorization, module scope and RBAC.
Telemetry & Feedback	Chat logs, ratings, failure flags, sequence depth	Tutor/admin analytics and system monitoring.
Vector Store	Curriculum chunks	Semantic retrieval of official course material.
Vector Store	Code repair patterns	Retrieval of relevant historical/common bug patterns.

A useful way to think about the two databases is: the relational database is the rulebook, while the vector database is the searchable library.

15. End-to-End Student Request Flow

The following sequence combines the architecture into one flow. This is the main process the team should understand.

- 1. Student submits a request:** The student selects a module and submits a question or broken code through the chat window.
- 2. API Gateway receives it:** The Gateway authenticates the request and passes the relevant identity and module information onward.

3. **Module scope is set:** The selected `module_id` is attached to the session and used to keep the request within the chosen module. Automatic population from official enrollment records is deferred for the current four-module, single-course scope.
4. **Intent is classified:** The Intent Agent determines whether the request is conceptual, debugging-related, or another supported intent.
5. **Socratic rules are applied:** The Scaffolding Engine determines how directly the AI should guide the student based on the current session state.
6. **Question is embedded:** The RAG Orchestrator converts the question into a vector using the embedding model.
7. **Course context is retrieved:** The Orchestrator performs a module-filtered semantic search against pgvector.
8. **Prompt is assembled:** The system combines tutoring rules, retrieved course material, relevant chat history, and the student's current request.
9. **Inference model generates a draft:** The selected open-weight model generates the tutoring response.
10. **Guardrail checks the draft:** The Guardrail Agent checks for solution leaks and other failures before delivery.
11. **Student receives the hint:** The approved response is displayed in the same chat window.
12. **Feedback is recorded:** The student's rating and relevant telemetry are stored for analytics.
13. **Dashboards update:** Tutors see only their assigned module data; the Admin can monitor system-wide performance.

16. Tutor & Admin Dashboards

16.1 Tutor Dashboard — Micro Level

The tutor dashboard is scoped to the tutor's assigned modules. It should help answer questions such as:

- Which topics are generating the most questions?
- Where are students receiving the most negative ratings?
- Which bugs or concepts appear repeatedly?
- Are students requiring unusually long hint chains?

16.2 Admin Dashboard — Macro Level

The admin dashboard provides global system oversight. It should help answer:

- Is the AI performing consistently across modules?
- Are negative ratings increasing across the whole system?
- Are guardrail/code-leak flags increasing?
- Are tutor-module mappings correct?
- Does the system need prompt, model, or retrieval changes?

17. Human Adjustment Cycle

The system is intended to support a feedback cycle rather than operate as an isolated AI tool.

20. Students use the tutor and provide feedback.
21. Telemetry records the interaction and relevant AI/system signals.
22. Tutors review module-specific trends and identify academic problem areas.
23. Admins review system-wide trends and identify technical or model-level issues.
24. Tutors can adjust teaching emphasis where students are struggling academically.
25. Admins can adjust prompts, model settings, retrieval configuration, or other system components when the AI is responsible for the problem.

18. Architecture at a Glance

The supplied architecture diagram can be understood in the following simplified structure:

CLIENT TIER

Student Interface → Tutor Dashboard → Admin Dashboard

ORCHESTRATION TIER

API Gateway → Intent Agent → Scaffolding Engine → RAG Orchestrator → Guardrail Agent

DATA TIER

Relational Store / Telemetry & Feedback ↔ Vector Store (Curriculum + Code Repair Patterns)

INFERENCE TIER

Unified Open-Weight Model via Local Ollama (Qwen3 4B)

The actual runtime flow is not simply a straight line: the request moves downward through validation, scaffolding, retrieval, and inference, then returns upward through guardrail auditing and delivery. Feedback then flows back into the telemetry layer and dashboards, creating a continuous improvement cycle.

19. Current Architecture Decisions

- The current system supports four modules within one course.
- The Student Interface uses a module dropdown and the selected `module_id` is attached to each chat session.
- Automatic assignment of a student's modules from official enrollment records is deferred for the current scope, but the data model should leave room for it later.
- There are three user roles: Student, Tutor, and Admin.
- Students and Tutors authenticate using their institutional DUT4life email accounts; the application handles role-based authorization after identity verification.
- Admins grant Tutor privileges and manage tutor-module assignments.

- Tutors are restricted to assigned modules through backend/database-level access control.
- The system supports both theory and coding modules and uses a Socratic tutoring approach rather than directly providing final solutions.
- The Scaffolding Engine tracks the student's progression through the tutoring process.
- RAG is used to ground responses in official course material.
- The vector store uses module metadata to prevent cross-module retrieval.
- PostgreSQL and pgvector form the planned hybrid data layer.
- The working embedding model is nomic-embed-text via Ollama.
- The working generative model is Qwen3 4B via Ollama.

20. Items Still to Be Finalized

The architecture and project scope have now resolved the earlier model, embedding, feedback-tag, authentication-direction, and deployment questions. The remaining items are implementation details that the current paper does not fully specify:

- Exact numerical thresholds and detailed implementation rules for Guardrail blocking/truncation. The paper defines the required behaviours but not the numeric cut-offs.
- Exact dashboard metrics, visualizations, and alert thresholds.
- How code-repair patterns will be collected, stored, reviewed, and maintained for modules where coding support is relevant.
- Exact DUT identity-provider configuration (for example, tenant/application registration details) required to enable institutional login.
- Future automatic module enrollment/assignment from official student enrollment data. This is intentionally deferred while the system remains limited to four modules in one course.

21. What Each Team Member Should Understand

Even if members are responsible for different components, everyone should understand the overall request path:

26. Student asks a module-specific academic question or submits a coding problem where relevant.
27. Gateway authenticates and routes it.
28. Intent Agent identifies the request.
29. Scaffolding Engine determines the tutoring behaviour.
30. RAG retrieves the correct course context.
31. Inference model generates the response.
32. Guardrail checks the response.
33. Student receives the hint.
34. Feedback and telemetry update the tutor/admin views.

The most important architectural boundary is that no single component should be responsible for everything. Authentication, tutoring rules, retrieval, generation, output checking, and analytics have separate responsibilities.

Appendix A — Key Terms

Term	Meaning
API Gateway	The backend entry point that receives requests, authenticates them, and routes them.
RBAC	Role-Based Access Control; controls what users can access based on their role and assignments.
Intent Agent	Classifies what the student is trying to accomplish.
Scaffolding Engine	Applies the Socratic teaching rules and tracks tutoring progression.
RAG	Retrieval-Augmented Generation; retrieves external/contextual information before generation.
Embedding	A numerical representation of text meaning used for semantic search.
Vector Store	A database/index containing embeddings and metadata used for similarity retrieval.
pgvector	PostgreSQL extension used to store and search vector embeddings.
Inference Tier	The part of the system where the generative AI model produces the response.
Guardrail Agent	Checks the model's draft before it is delivered.
Telemetry	Recorded system and interaction data used for monitoring and improvement.
Module ID	Identifier used to associate users, sessions, curriculum, and permissions with a specific module.
Session ID	Identifier used to keep a particular student conversation and its telemetry together.